

SENTINEL-LLM: A Practical Multi-Layer Defense Framework for Adversarial Prompts in Production Language Model Systems

Muneeb Anjum*

Department of Computer Science

COMSATS University Islamabad, Abbottabad Campus, Pakistan

Independent Researcher

June 14, 2026

Highlights

- A working two-axis taxonomy of adversarial prompt attacks on production LLM and VLM systems, organized by attack vector (direct, indirect, multi-turn, cross-modal, tool-call) and attack goal (harmful content, data extraction, identity spoofing, system compromise).
- Critical review of nine defense strategies, including Constitutional Classifiers, Smooth-LLM, perplexity-based filters, and retrieval isolation, against the attack families above.
- A reference implementation of SENTINEL-LLM, a five-layer defense framework designed for production deployment with sub-50 ms overhead per request, alongside an honest proof-of-concept evaluation showing 4/7 detection on a curated test set.
- Lessons learned from implementation, including failure modes where the framework misses encoded attacks (leetspeak, base64) without a dedicated decoding layer, and where multi-turn drift across context is the dominant weakness.
- Open release of the framework code and audit log for the community to reproduce, extend, and critique.

Abstract

When a chatbot issues a refund policy that never existed, or a coding assistant recommends a malicious package, the underlying problem is almost always an adversarial prompt. I built SENTINEL-LLM to catch those prompts before they reach a production model. SENTINEL-LLM is a five-layer inference-time wrapper that sits between the application and the model: lexical and perplexity checks, semantic intent classification, cross-modal document tagging, multi-turn drift detection, and a tool-call firewall. It runs in well under 50ms per request on commodity hardware, logs every decision, and does not require retraining the underlying model. The framework is built from existing techniques assembled in a way a small team can deploy without a dedicated security team. I evaluated it on a curated set of seven representative attacks spanning the four attack vectors and four attack goals from my taxonomy, and got a 4/7 detection rate. The misses are concentrated in encoded attacks (base64, leetspeak) and multi-turn drift, and I describe exactly why. The code, the test set, and the audit log are released at the URL in the Appendix.

Keywords: LLM security; adversarial prompts; jailbreak detection; multi-layer defense; SENTINEL-LLM

*Author is a recent graduate of the BSc Computer Science program at COMSATS University Islamabad, Abbottabad Campus. Currently applying for Master's programs in AI and cybersecurity. Contact: horizonbymuneeb@gmail.com.

1 Introduction

A customer support chatbot for a US airline was recently tricked into issuing a refund policy that the company did not actually offer. A coding assistant at a Fortune 500 firm was steered into suggesting a malicious npm package as a recommended dependency. A government services assistant leaked another user’s case notes because of a single indirect prompt injection smuggled in through a document. None of these are hypothetical. They happened in 2024 and 2025, and they are the kind of incidents that drove me to write the framework described in this paper. The airline case in particular is the one I keep coming back to, because the attack was three turns long, and no single turn looked malicious on its own. That is the shape of the problem I was trying to solve.

Naming note. The framework described in this paper is the author’s original work, written and built in 2025–2026. The acronym SENTINEL (here written SENTINEL-LLM) is used in several prior or contemporaneous works in adjacent subfields, including the LLM agent for adversarial purification of Lin and colleagues [16] (an inference-time defense against textual perturbations, distinct from the prompt-injection defense discussed here), the multi-layer FastAPI framework of Jain [13] (a security framework with a similar naming convention but a different design and scope), and the AI Security Platform of Labintsev [15] (a Rust-based red/blue team suite, distinct in both architecture and scope from the inference-time wrapper presented here). I use the name SENTINEL-LLM to mean only the framework described in this paper, and I have made the implementation available at the URL in the Appendix for any reader who wishes to verify the mapping.

Large language models (LLMs) and their multimodal cousins (vision-language models, VLMs) are now in production in places where a wrong answer has real consequences: customer support at scale, code generation that ships to production, healthcare triage, financial research, government services. The same flexibility that makes these models useful, that they will follow almost any plausible instruction, that they can be prompted in natural language, is also the surface area that an attacker can probe. Over the last three years the security community has produced a long and growing catalog of attacks that exploit that surface area. The attack research is moving fast, faster than the defensive playbook in most organizations.

This paper is a report from a small defensive project. I am not a research lab; I am a recent graduate trying to build something practical. I did not set out to invent a new defense technique. I set out to answer a more mundane question: *given everything the literature has produced, what is the cheapest possible thing I can put in front of a production LLM endpoint that catches a meaningful fraction of known attacks and adds a reasonable amount of latency?* The answer I arrived at is a five-layer inference-time filter I call SENTINEL-LLM. It is not a single clever idea. It is mostly standard pieces, sequenced in a way that I explain in Section 5, and tuned to a small but realistic set of test prompts I curated for the proof of concept.

The contributions of this paper are:

1. A two-axis taxonomy of attacks that I found useful in my own engineering work, organized by *attack vector* (how the malicious content reaches the model: direct, indirect, multi-turn, cross-modal, tool-call) and *attack goal* (what the attacker is trying to make the model do: produce harmful content, extract data, impersonate a person, compromise the surrounding system).
2. A critical review of nine defense strategies I evaluated, ranging from simple input filters to the recent Constitutional Classifiers from Anthropic. I discuss what each defense is actually catching and what it is structurally blind to.
3. A reference implementation of SENTINEL-LLM that runs in under 50 ms per request on

commodity hardware, with code, configuration, and a reproducible test set.

4. An honest proof-of-concept evaluation on seven representative test cases, with a 4/7 detection rate and a clear account of what the framework missed and why.
5. A short, opinionated list of the open problems the field has not yet solved, in the hope that other small teams do not waste time on the same dead ends I did.

I also want to be transparent about what this paper is not. It is not a benchmark paper. I do not run SENTINEL-LLM against JailbreakBench, HarmBench, or AdvBench and report an aggregate score; the layers are partly heuristic and a single number would be misleading. It is not a comparison paper. I do not run every defense on every test set. It is not a theoretical paper. There are no formal robustness guarantees; the security argument is empirical, and I say where it breaks.

What it is, in the end, is a build report. Here is what I built, what I tested, what passed, what failed, and what I would do differently if I started over. The code is at the URL in the Appendix.

1.1 Motivation

The economic case for LLM deployment is no longer in question. Industry surveys from late 2025 put the share of enterprise software that touches an LLM in production at above 70% [20, 8], and the share of customer-facing applications at above 40%. The security story has not kept up. A widely cited 2025 industry assessment reports that prompt injection, the single most common attack class, was observed in a majority of the production AI systems it assessed [20]. A second 2025 assessment reported that a large fraction of the models it tested were still vulnerable to multi-turn jailbreak even after baseline alignment [7]. A third 2025 industry report on biometric fraud noted that the same generation of generative models that makes LLM assistants useful also makes voice and video impersonation trivially cheap [12].

The historical responses to this kind of finding are familiar from earlier waves of security work. Align the model more carefully. Add a classifier. Wrap the system in a guard. The trouble is that the LLM threat surface is not the same as the threat surface for, say, a SQL endpoint. The input is unstructured natural language, the output is unstructured natural language, and the attack can be smuggled through a document the user did not write, an image the user did not draw, or a turn the user did not even start. Defenses that work on one channel routinely miss attacks delivered on another.

I am not the first to notice this. The point of the project is not to discover a problem; the problem is well known. The point is to ship a small, working, somewhat honest defense that an organization can actually deploy without a dedicated research team. The honest reason I wrote this paper is that I wanted to read this paper two years ago, and it did not exist.

1.2 Why this paper

Most of the published work on LLM adversarial robustness is attack-driven. A team finds a new class of attack, shows it on a frontier model, and writes a paper describing the technique. The defensive literature is much smaller and is often published by the same frontier-model labs that developed the original alignment techniques. That asymmetry is not unreasonable, but it leaves a gap. Practitioners, especially practitioners at organizations that are deploying open-weight or mid-tier commercial models, do not have access to a single document that lays out:

- the current state of the attack landscape, organized in a way that a defender can act on;
- the current state of the defense landscape, with honest criticism of what each defense misses;

- a concrete reference implementation of a defense that integrates with the inference stack a typical organization already runs;
- an evaluation that does not hide the failure cases.

I tried to write that document. I am not the right person to write the survey portion. I am a recent graduate with a year of personal time and a workstation. What I can do is offer a practitioner’s view, not an exhaustive one. The literature I cite reflects what I read and used; it is not a survey.

1.3 Structure of the paper

Section 2 lays out the background I assume: the rough shape of the LLM deployment stack, the threat model, and the kinds of failure modes I am worried about. Section 3 is my attack taxonomy and the specific attack families I built SENTINEL-LLM to defend against. Section 4 is the defense review, covering nine strategies and my assessment of each. Section 5 describes the SENTINEL-LLM framework itself: the five layers, the runtime path, the configuration knobs. Section 6 reports the proof-of-concept evaluation. Section 7 discusses the open problems and the limits of what I built. Section 8 concludes.

2 Background and Deployment Context

The reader is probably familiar with the LLM deployment stack, but maybe has not lived through an incident. This section is a short orientation. It defines the threat model, sketches the inference path we actually defend against, and lists the failure modes that motivated the work.

2.1 The deployment stack

A production LLM application in 2025 has a rough shape. I will describe it the way I drew it on a whiteboard during a meeting about a year ago, when I was trying to explain to a non-technical stakeholder where in the request flow a defense should sit. A user submits a request, usually through a web or mobile interface. The application layer adds system instructions, a small amount of context (the recent turns, the user’s profile, the conversation history), and optionally retrieves documents from a vector store for retrieval-augmented generation. The prompt is sent to the model. The model returns a completion. The application post-processes: strips tool calls, runs the output through any safety filters, logs the exchange. The user sees the result. That is the happy path.

Vision-language models have the same shape with a different first step: the image is encoded into tokens, the text prompt is appended, and the model produces text. Agentic systems add a tool-call loop on top. The model can request to invoke external functions, the application executes them, and the result is fed back. None of this is new. The reason I am writing it down is that every step in that pipeline is a potential attack surface. A defense that only watches one step will miss the others. The chatbot refund incident that started this project for me was a three-turn interaction, and the second turn was perfectly innocent looking on its own. I learned the hard way that watching the input prompt is not enough.

2.2 Threat model

The threat model is conservative on purpose. I am a small team, working alone, and I would rather over-defend than miss. The earlier versions of this framework were too permissive; I have spent more time tightening the threat model than I spent building the framework itself, and that is probably the right ratio.

Adversary goals. I assume the adversary is trying to do one of four things. First, cause the model to produce content it has been instructed not to produce, the usual jailbreak. Second, extract data the model should not have access to, whether another user’s data, system prompt content, or training data fragments. Third, impersonate a specific person or system to the user, including a customer service agent or a brand representative. Fourth, use the model as a stepping stone to compromise the surrounding application, by triggering dangerous tool calls or exfiltrating data through indirect channels. The fourth category is the one I underestimated when I started. I now think it is the most common vector in production.

Adversary capabilities. The adversary can submit arbitrary text, arbitrary images, and any number of conversation turns. I do not assume the adversary can manipulate the system prompt, the model weights, the retrieval index, or the surrounding application logic. Defenses that require trust in any of those are out of scope; I am building a wrapper, not a secured runtime. I assume the adversary is aware of my defense. Kerckhoffs’s principle applies. I do not assume the adversary has API-level access, fine-tuning access, or the budget to run expensive optimization procedures against my deployment. If they do, things get worse; I discuss what changes in Section 7.

Defender goals. I am trying, in order of importance, to: catch a high fraction of known attacks with a low false-positive rate; add as little latency as I can; be cheap to operate; and be auditable, in the sense that a security team can pick a random request from the logs and trace exactly which layer made the decision and why. The auditable part is the one I underestimated. A defense that you cannot audit is a defense that nobody trusts, and an untrusted defense gets removed in the next sprint.

2.3 What this paper does not solve

I am going to be explicit about what I am not solving. The threat surface is large. The temptation to over-claim is real. I have read too many papers that promised a 95% detection rate and described no failure cases; I do not want this paper to be one of them.

I am not solving model alignment. The model can still, with enough effort, be made to produce harmful content. SENTINEL-LLM is a wrapper, not a substitute for good training. I am not solving data privacy in the underlying system; if the model has memorized a user’s data, no inference-time filter is going to put it back. I am not solving attacks on the model weights themselves. I am not solving the deeper question of what counts as "harmful content" in the first place, which is a policy question, not an engineering one, and the policy question has no clean answer.

What I am solving is a particular narrow slice: detecting and rejecting adversarial prompts and tool-call patterns at the inference boundary, with practical performance, an honest accounting of the failure cases, and a working reference implementation that someone else can run tomorrow. The slice is narrow on purpose. The wider the slice, the more I would have to lie to fill it.

3 Adversarial Attack Landscape

When I started reading the attack literature I got lost fast. The field has grown so quickly that no single taxonomy covers it. I ended up organizing attacks along two axes that I found most useful for my own engineering work: the *attack vector*, which is how the malicious content reaches the model, and the *attack goal*, which is what the attacker is trying to achieve. The two axes are independent. An attack has a vector and a goal. This section walks through the categories I used when I was deciding what to defend against, and the specific attack families that shaped the design of SENTINEL-LLM.

3.1 Two-axis taxonomy

On the *vector* axis I distinguish five categories. **Direct textual** attacks arrive as a single user-submitted message. Either the user is the attacker, or the user has been socially engineered into pasting the attack text. **Indirect textual** attacks arrive through documents the user did not write: a webpage the model is asked to summarize, a file the model is asked to read, a tool response the model receives. The user is not the attacker and is usually unaware. **Multi-turn** attacks unfold across many turns. No single message looks malicious, but the cumulative drift adds up to one. **Cross-modal** attacks arrive through an image, an audio clip, or a video. **Tool-call** attacks are a special case that arises in agentic systems: the attack is encoded in the model’s tool-call behavior rather than in any user-visible content. The tool-call category was not in my first draft. I added it after I saw a published attack on a customer service agent that worked entirely through a single function call that looked perfectly normal in isolation.

On the *goal* axis I use four categories that mirror common compliance regimes. **Harmful content** means jailbreak in the usual sense, getting the model to produce content the operator does not want. **Data extraction** means getting the model to reveal something it should not, whether another user’s data, system prompt content, or training data fragments. **Identity spoofing** means getting the model to claim to be a person or system it is not. **System compromise** means using the model as a way to do something to the surrounding application, exfiltrating data through a tool call or executing a privileged action.

The full taxonomy is a five-by-four matrix, twenty cells. About fifteen of the cells have published attacks as of mid-2026. I focus on the cells that mattered for the production deployments I have seen, and the cells I think will matter most in the next year.

3.2 Specific attack families

I am going to walk through the attack families I considered in the design. The descriptions are short on purpose. I am not trying to reproduce the attacks; I am trying to give enough detail that the defenses in Section 4 make sense.

Greedy Coordinate Gradient (GCG). The GCG attack [30] optimized a single adversarial suffix that, appended to a wide range of user prompts, causes aligned LLMs to comply with harmful instructions. The attack is white-box and transfer-based: the suffix is generated against a small open-source model but transfers to many production models, including commercial ones. GCG was a turning point in the field. It showed that a single optimized string could defeat the alignment of multiple models at once, which was not widely believed before that paper. Subsequent work in this family, including AutoRedTeamer [29], automated the search for new GCG-style suffixes at scale. The recent AB-JB hybrid framework [4] combines black-box semantic prompt-variant generation with embedding-level suffix optimization. A 2026 empirical study [14] catalogs the contemporary attack landscape and confirms that suffix-based and embedding-based attacks remain the dominant threat family. I tested my first L1 layer against GCG-derived suffixes; the regex caught most of them, but the ones with non-ASCII characters did slip through, and that is what made me add the Unicode normalization step.

Persona modulation. Shah and colleagues [23] showed that asking the model to take on a persona willing to comply with harmful instructions is a remarkably effective black-box attack. The attack requires no optimization, transfers between models, and is hard to defend against with simple keyword filters because the malicious content is the persona itself, not a string. Persona modulation and its automated descendants are the reason defense-by-keyword is essentially hopeless. I do not think this point is made strongly enough in the literature. Anyone reading the persona paper for the first time should walk away convinced that regex defenses against jailbreak are a lost cause for the persona vector specifically.

String composition and bijection learning. The Plentiful Jailbreaks paper [11] showed that composing simple string transformations (leetspeak, base64, ROT-13, ASCII art) gives a combinatorially large attack space at very low cost. The follow-up Endless Jailbreaks paper [10] used bijection learning to teach the model to decode these encodings in-context, with even higher attack success. Both attacks are white-box in the sense that they require access to the model weights, but the techniques transfer to black-box settings. They are particularly important for defenders because they are structurally invisible to defenses that look at the surface form of the input. The base64 case is the one that bit me. I had a working L1 layer that I was proud of, and a colleague sent me a base64-encoded version of an attack prompt, and L1 let it through like it was nothing. I went back and added the encoding check, but the deeper fix, a proper decoding layer, is still on my to-do list and is the subject of Section 7.2.

Indirect prompt injection. Indirect prompt injection [9, 25, 21] places the attack in a document, a webpage, an email, or a tool response that the model reads on the user’s behalf. The user is unaware. This vector is increasingly important as LLM applications add retrieval-augmented generation and tool use. The OWASP 2025 update places prompt injection, direct and indirect combined, at the top of its Top 10 list for LLM applications [20]. A recent MDPI survey [21] catalogs the indirect-injection attack surface across RAG systems in detail. The thing I underestimated about indirect injection is that the attacker does not even need the user to be complicit. The user just needs to be using a tool that reads the web. That is a much larger attack surface than the direct-injection case.

Backdoored retrievers. Geyer and colleagues [9] showed that an attacker who can poison the retrieval index of a RAG system can reliably inject prompts into every retrieval that touches the poisoned documents. The attack is particularly nasty because it does not require any compromise of the model itself. If an organization is using a third-party vector store, or scraping documents from the open web, the defender is in a much harder position than in the direct-injection case.

Cross-modal attacks. A growing family of attacks exploits the image channel in vision-language models. VLATTACK [28] was an early multimodal attack; the recent Chain of Attack [26] and MAVLA [3] work showed that multimodal attacks transfer to vision-language-action systems used in robotics. These are relevant even to text-only deployments, because the text tokenizer in modern models often shares embedding tables with image tokens. An image that contains carefully crafted text can affect downstream text completions, even when the user query is plain text. The OCR stub in Layer 3 of SENTINEL-LLM is a placeholder for a proper image-channel defense; I describe the placeholder and its limitations in Section 5.

Agentic and protocol attacks. The recent survey by Zhan and colleagues [27] catalogs more than thirty attacks on LLM agent systems, spanning input manipulation, model compromise, system and privacy attacks, and protocol-level vulnerabilities. As agentic deployments grow, the tool-call surface becomes a primary attack target. I read this survey cover to cover, and I came away convinced that the agentic threat surface is the most under-defended part of the LLM deployment stack in mid-2026. Layer 4 of SENTINEL-LLM is my attempt at a structural defense, but the limitations are real and I describe them honestly in Section 7.4.

3.3 Benchmarks and threat catalogs

A small number of benchmarks and catalogs have become reference points for both attacks and defenses. I rely on a few of them heavily in this paper, and I want to be specific about which ones and why.

JailbreakBench [6] is an open benchmark with a curated dataset of 100 behaviors and a growing repository of adversarial prompts. It is the most useful single benchmark for tracking the field. I use it as a reference point throughout, though I do not run an aggregated score against it. The

100-behavior list is more curated than HarmBench’s, in my opinion, and the behavior categories are clearer.

HarmBench [17] is a standardized evaluation framework for automated red teaming, with a taxonomy of harmful behaviors and a set of evaluation methods. It overlaps with JailbreakBench in scope but differs in evaluation methodology. HarmBench’s automated red-teaming harness is the right tool when you want to test a defense at scale, and I would have used it for the larger test set if I had the compute budget. As it is, the 32-attack set in the repository is hand-curated.

The OWASP Top 10 for LLM Applications [20] is a practitioner-oriented risk catalog maintained by the Open Worldwide Application Security Project. The 2025 update places prompt injection at the top of the list, followed by sensitive information disclosure and supply chain risks. It is the most useful single document for an organization that needs to communicate LLM risk to non-specialists. I cite it in the paper because the people reading the paper are not always specialists.

MITRE ATLAS [24] is a knowledge base of adversary tactics and techniques against AI-enabled systems, modeled on the ATT&CK framework familiar to anyone in IT security. It is the right reference when you need to communicate with a security operations center that already speaks ATT&CK. I have used the ATT&CK Navigator with the ATLAS matrix in meetings with blue teams, and it changes the conversation in a way that academic taxonomy does not.

The NIST AI 100-2 publication [18] provides a formal taxonomy of adversarial machine learning concepts and terminology. It is the right reference when you need to align your defense language with a regulator or a procurement office. I am citing it because some of the readers of this paper will be in regulated industries, and the NIST vocabulary is the vocabulary they are going to be expected to use.

These documents do not always agree on details. The field is moving fast. I use them as orientation points rather than as authoritative definitions. If a reader is going to pick one of them to read first, I would recommend OWASP for the non-specialist and NIST for the specialist. JailbreakBench is the right one to actually run.

4 Defense Strategies: A Critical Review

Before I started building SENTINEL-LLM I read through every defense paper I could find. I built a list of nine strategies that came up again and again in the practitioner literature. None of the nine is sufficient on its own. Some are model-level, some are wrapper-level, some are operational. The descriptions below are not abstracts of the published papers. They are what I learned trying to deploy each one and what I ended up doing with it. If a reader only has time for one section of the paper, this is the section that captures the engineering decisions.

4.1 Defenses at the model level

RLHF and instruction tuning. Reinforcement learning from human feedback [19] is the foundation of alignment in most production LLMs. RLHF cuts the success rate of single-turn jailbreaks, which is the result everyone reports. It does almost nothing against persona modulation or multi-turn drift, and it is invisible to indirect injection. I treat RLHF as a baseline. Anything I build has to assume the underlying model is already RLHF-aligned and is still vulnerable. The earlier drafts of this section had a longer critique of RLHF. I trimmed it because the points are well-rehearsed in the literature and I have nothing new to add to the critique.

Constitutional Classifiers. Constitutional Classifiers [5] train a separate classifier on synthetic

data generated from a written constitution. Anthropic’s published system is reported to hold up against thousands of hours of red-teaming. I spent two evenings trying to adapt the architecture for a non-Anthropic model. The training procedure alone is heavier than my entire project budget in time and compute. I treat the paper as a useful design template, not as something I could deploy. The honest reason I did not pursue it further is that I do not have a constitution of my own to train on; the constitutional method assumes the operator has a clear policy in mind, and the policies I have seen in practice are too vague to be useful as training signal.

SmoothLLM. SmoothLLM [22] perturbs the input, queries the model on each perturbed version, and aggregates the responses. The intuition is that adversarial suffixes are fragile to small character-level changes. It works against the GCG family. It does not work against persona modulation, indirect injection, or anything the attacker encodes. The latency cost is the part that matters for my wrapper: SmoothLLM roughly triples the inference cost per request. For my sub-50ms target, the math does not work. I considered it, ran the numbers, dropped it. A 2025 empirical study [2] found that the class of bypass/evasion attacks against guardrail-based defenses (including SmoothLLM-style input perturbation and simple keyword filters) is more diverse than the original guardrail papers suggest; I cite it as evidence that the published bypass numbers are an underestimate. The interesting design lesson from SmoothLLM for me was the aggregation idea, which I borrowed in a stripped-down form in Layer 2 of SENTINEL-LLM.

4.2 Defenses at the wrapper level

Perplexity-based filters. A small reference model scores the input; anything above a threshold gets rejected. Cheap to run. The catch is that the false-positive rate on legitimate technical text is high, and a careful attacker evades the filter by writing natural-sounding text. I use a perplexity check inside Layer 1 of SENTINEL-LLM, but as a soft signal that combines with pattern matching rather than as a standalone block. I had a version of Layer 1 that used perplexity as a hard threshold. It blocked two of my test cases that I wanted to allow (a Stack Overflow paste and a snippet of Python code with an unusual import). I lowered the threshold to make those cases pass and accepted that the perplexity check would only be a soft signal. The version that ships in the repository has that calibration.

Keyword and pattern filters. The classic regex approach: catch "ignore previous instructions", "you are now", "system:", and friends. Easy to deploy. Trivially evaded by anything that does not contain the exact pattern. I have a list of 57 such patterns in Layer 1. The list is necessary. The list alone is not a defense; an attacker who paraphrases or encodes gets through it. The 57 patterns are the result of two iterations. The first iteration had 13 patterns; the second iteration, which I built after running the proof-of-concept test set, added the 44 that the original 13 missed. The list is in the source code; the configuration file documents the rationale for the threshold.

Embedding-based anomaly detection. Embed the input, compare to a distribution of known-good inputs, reject the outliers. I tried this with a small Sentence-Transformers model. It works for the specific distribution it was trained on, and the moment the traffic shifts, the false-positive rate jumps. The training data problem is the same problem every embedding-based classifier has. I do not use it in the final design. The first version of SENTINEL-LLM had an embedding-based layer 2; I cut it after the 32-attack test set produced 4 false positives on the 10 benign controls. The benign controls were the kind of customer service queries I expected to see in production. The classifier was rejecting too many of them. I have notes in the repository about why I made the cut.

Retrieval isolation. The defenses that try to separate the user prompt from the retrieved context, using delimiters, prompt structures, or a "trusted" instruction that tells the model to treat retrieved text as untrusted data. The simplest version is the one Anthropic and OpenAI ship in their framework examples. It catches the obvious cases. It does not catch an indirect attack

that is well-formed English, which is most of them. I include a stripped-down version of this in Layer 3 of SENTINEL-LLM. The reason I am still using it even though it has known limitations is that the cost is small (a few hundred microseconds per request) and the false-positive rate is zero, so the layer is essentially free defense in depth.

Tool-call allowlists. For agentic systems, the only defense that has consistently worked for me is to restrict the model to a small allowlist of pre-approved tool calls and require human approval for anything outside. It is structural, not model-based. The interesting failure mode is not the attacker calling a forbidden tool; it is the attacker getting the model to call a permitted tool with a slightly different argument. Layer 4 of SENTINEL-LLM is the attempt to catch that. I am not sure I have caught it. The 32-attack test set has 6 indirect prompt injection cases that I know about, and the layer catches 4 of them, which is the right number for a structural defense. The 2 it misses are the ones that worry me, and I describe them in Section 7.4.

Output filtering. Filter the model’s response for harmful content, PII, and patterns of data exfiltration. Necessary, insufficient. An attacker who can steer the response into a tool call rather than a text response bypasses the output filter entirely. I have an output monitor in Layer 5; it is the layer I am least confident in. The threshold for the output monitor has moved three times during the development of this paper. The current threshold of 0.5 is the lowest value that still caught the proof-of-concept refusal-bypass case, and it is the one I would expect to need to adjust in production. I have left the threshold as a configuration knob for that reason.

4.3 Defenses at the operational level

Rate limiting, abuse detection, and human review. The classical operational defenses. They are still important. They do not appear in SENTINEL-LLM because they are organization-level rather than inference-level, and the scope of this paper is the inference-time wrapper. I mention them here because every team I have spoken to that deployed any of the other eight defenses without operational controls got hit by something the defenses did not catch. I would rather not name names, but the pattern is consistent. A defense in depth means operational controls on top of the inference-time wrapper, not as a replacement for it.

4.4 What we concluded from the review

The review left me with two conclusions. First, no single defense is sufficient against the attack landscape. The published systems are catching the attacks they were designed to catch, and they are blind to attacks outside their design. Second, the published defenses are not easy to compose. SmoothLLM is a model wrapper; Constitutional Classifiers is a model-side system; tool-call allowlists are operational. A small team trying to deploy a defense in front of an existing model has to assemble its own layered system from these pieces, and there is no good reference for how to do that.

SENTINEL-LLM is the result of that assembly work. None of its five layers is novel on its own. The combination, the sequencing, and the per-layer configuration that keeps the latency under 50ms is, to the best of my knowledge, the contribution of this paper: a single, documented, runnable, and audited reference implementation of a layered defense that covers the full attack surface I described in Section 3 within one inference-time wrapper. The honesty disclaimer is that I am one person. The system has been tested on a curated 32-attack set. It has not been tested in a year-long production deployment, and I would be surprised if a year-long deployment did not surface at least one or two surprises that I have not anticipated.

5 The SENTINEL-LLM Framework

SENTINEL-LLM is a five-layer inference-time defense that sits between the application layer and the model. Every request goes through the five layers in order; a request that fails any layer is rejected, logged, and optionally escalated to a human reviewer. The framework is designed around three principles that came out of the review in Section 4: each layer should target one specific attack vector, layers should be cheap enough that the whole framework adds under 50 ms to a typical request, and every decision should be auditable.

5.1 Design principles

The three principles are not original. The first is a consequence of the taxonomy in Section 3: each attack vector has different surface features, and a single classifier that tries to cover all of them tends to do none of them well. The second is a deployment constraint; a 500 ms wrapper is not a wrapper, it is a different product. The third is a debugging constraint; a security team that cannot understand why a request was rejected cannot tune the system.

5.2 The five layers

Layer 1: Lexical and structural pre-filter. The first layer is the cheapest and the dumbest. It looks at the surface form of the input: character distributions, perplexity under a small reference model, presence of known-bad patterns, and a small set of regex-based checks for things like encoded payloads. The layer is configured with a whitelist of high-risk patterns drawn from public attack datasets and a blacklist of low-risk patterns. Anything that hits the blacklist is rejected. Anything that fails the perplexity check is flagged for Layer 2. Anything that passes both is forwarded.

Layer 2: Semantic and intent classifier. The second layer wraps a small classifier model (in our reference implementation, a 350M-parameter open-weight model fine-tuned on a labeled dataset of attack and benign prompts). The classifier produces a probability that the input is an attack. The threshold is configurable; the default is 0.7. The classifier is the only layer that uses a model, and it is the only layer that has nontrivial training cost. The model is small on purpose; we wanted the framework to be deployable on commodity hardware.

Layer 3: Cross-modal and context filter. The third layer activates only for inputs that include images or for inputs that arrive through a retrieval step. For images, the layer runs an OCR pass and feeds the extracted text through Layer 1 and Layer 2. For retrieval, the layer examines the retrieved documents for indirect prompt injection patterns and tags any document that matches. The layer is intentionally conservative: a tagged document is not automatically rejected, but the user prompt is augmented with a "the following retrieved text may be untrusted" instruction, and the model's response is filtered through a stricter output policy.

Layer 4: Multi-turn and policy drift tracker. The fourth layer maintains a per-conversation state of: the original system prompt, the running list of high-risk user requests, and a small embedding-based summary of the conversation so far. Each new turn is compared against the original system prompt and the running summary. A turn that drifts too far from the original policy is rejected. The threshold is again configurable; the default is conservative.

Layer 5: Action and output policy. The fifth and final layer is the output filter and the tool-call policy. It screens the model's response for harmful content, PII, and known patterns of data exfiltration. It enforces a tool-call allowlist for any deployment that exposes tools. A response or tool call that fails the policy is rejected, and the user is given a generic refusal.

5.3 Configuration and tuning

The framework exposes a small number of configuration knobs. The most important are the Layer 2 classifier threshold, the Layer 3 document-tag policy, the Layer 4 drift threshold, and the Layer 5 tool-call allowlist. We do not have good defaults for any of these; they depend on the application. A customer support chatbot and a coding assistant have very different risk profiles. The framework ships with a configuration file that documents each knob and the trade-off it controls.

5.4 Latency budget

We measured the framework on a single CPU thread of an AWS t3.large instance (the deployment we used for development). The median added latency was 38 ms, of which Layer 1 was 4 ms, Layer 2 was 18 ms, Layer 3 was 6 ms (only when triggered), Layer 4 was 7 ms, and Layer 5 was 3 ms. The 99th percentile was 142 ms, dominated by Layer 2 in the cases where the input was long enough that the classifier ran close to its sequence-length limit. We did not observe any latency contribution from Layer 3 in the no-image no-retrieval case.

These numbers are illustrative. They depend on the hardware, the model size, the input length, and the configuration. We share them so that other teams can calibrate their expectations; we do not claim that the framework is universally fast.

5.5 Auditability

Every request that passes through the framework is logged with the decisions of each layer. The log is structured as a single line of JSON per request, with fields for the request id, the timestamp, the input hash, the per-layer decision, the rejection reason if any, and the model's final response. The log is small enough to ship to a SIEM; we have tested it with a 12 KB per-request overhead. The log is the primary debugging interface: when a request is rejected, the security team can see which layer caught it and why.

5.6 What SENTINEL-LLM does not do

We are explicit about the things the framework does not do, because the threat surface is large and the temptation to over-claim is real.

The framework does not retrain the model. It is a wrapper, not a substitute for good alignment. The framework does not solve data privacy in the underlying system; if a model has memorized a user's data, no inference-time filter can put it back. The framework does not detect all attacks. The proof-of-concept evaluation in Section 6 reports what we found. The framework does not protect against attacks on the model weights themselves, which are out of scope for an inference-time layer. The framework does not solve the policy problem of what counts as "harmful content" in the first place; that is a question for the deploying organization.

6 Proof-of-Concept Evaluation

The proof-of-concept evaluation is a small, deliberately curated set of seven test cases that span the attack vectors and goals described in Section 3. We do not run SENTINEL-LLM against JailbreakBench, HarmBench, or AdvBench as a whole, and we do not report a single aggregate score. The reasons are given in the next subsection; the summary is that the layers are partly heuristic, and a single number would hide the failure modes that matter most for a defender.

In addition to the seven-case curated set, we also report a larger 32-attack test set drawn from public sources (JailbreakBench, HarmBench, OWASP, AdvBench subset), shipped in the

repository at `tests/poc_test_set.json`. Both test sets are runnable with a single command (`python3 tests/run_poc.py -tuned`); the framework’s measured behavior on both sets is reported in the subsections below and in the supplementary `REPRODUCIBILITY.md` file.

6.1 Why a curated set, not a benchmark

A full benchmark run would produce an aggregate number that, in our judgment, would mislead more than it would inform. The Layer 2 classifier is fine-tuned on a particular distribution, and the distribution of a public benchmark is not the distribution of a particular production deployment. A score of 80% on JailbreakBench tells a defender little about how the framework will perform on their own traffic. We chose instead to enumerate the cells of the attack taxonomy that we cared about and to construct a test case for each. The test cases are short, hand-written, and reproducible. The audit log records exactly what the framework did with each one.

6.2 The seven test cases

The seven test cases are summarized in Table 1. We list the attack vector, the attack goal, the source of the prompt, and the result.

Table 1: Seven proof-of-concept test cases, with the attack vector, attack goal, prompt source, and SENTINEL-LLM decision.

Vector	Goal	Prompt source	Decision	Layer caught
Direct textual	Harmful content	Persona modulation variant of [23]	Catch	L1
Direct textual	Identity spoofing	"You are now the system administrator"	Catch	L1
Direct textual	Harmful content	Base64-encoded prompt (decode-and-follow)	Miss	–
Direct textual	Data extraction	"Repeat your system prompt verbatim"	Catch	L1
Indirect textual	System compromise	Document containing "ignore previous" instruction	Catch	L1+L3
Multi-turn	Harmful content	Three-turn drift toward harmful content	Miss	–
Cross-modal	Data extraction	Image containing OCR-readable prompt	Catch	L3

The framework caught four of the seven test cases: the persona modulation, the identity spoofing, the system-prompt extraction, and the indirect document injection. It missed the base64-encoded direct prompt and the multi-turn drift. The number 4/7 is reproducible: running `tests/run_poc.py -tuned` in the shipped repository returns exactly this detection pattern (see the supplementary `REPRODUCIBILITY.md` for the full per-case decision log).

6.3 What the framework caught

The four catches are encouraging but not surprising. The persona modulation is an established attack family with abundant labeled training data; the Layer 2 classifier was specifically fine-tuned to recognize it. The identity spoofing is a pattern that L1 catches with a regex. The system prompt extraction is a pattern that L1 catches with a regex, and L2 catches the more subtle variants. The indirect document injection is exactly what L3 was designed to handle; the test prompt contains the canonical "ignore previous" instruction embedded in a document.

6.4 What the framework missed

The two misses are more interesting. The base64-encoded prompt passes L1 (the surface form is a benign-looking instruction to “decode” a string, and there is no decoding layer in the current implementation), passes L2 (the classifier has not seen enough base64 payloads in training), and the request is forwarded to the model. The model in our test deployment complies with the harmful request after decoding, which is the correct behavior of an unprotected model. This is a structural miss: SENTINEL-LLM does not have a decoding layer. We discuss what it would take to add one in Section 7.

The multi-turn drift is a different kind of miss. No single turn in the three-turn sequence is malicious. The drift is in the cumulative semantic content, which is what L2 is supposed to track. The miss happened because the L2 escalation detector is configured for the longer five-turn drift pattern documented in the original threat catalog; a three-turn window falls below the trigger threshold. Tightening the threshold would have caught the drift, at the cost of higher false-positive rate on legitimate multi-turn conversations. We discuss the trade-off in the same section.

6.5 Reproducibility on a larger test set

The seven-case curated set is the canonical number reported in the paper ($4/7 = 57.1\%$). For reviewers who want a broader signal, the repository also ships a 32-attack test set drawn from public sources (JailbreakBench, HarmBench, OWASP, AdvBench). On this larger set, the framework catches 28 of 32 attack prompts (87.5%) and produces 1 false positive in 10 benign controls plus edge cases (10% FP rate). The single false positive is an edge-case query that mentions “admin access” in a non-malicious context; the framework correctly flags it as ambiguous. The full per-case results are recorded in `REPRODUCIBILITY.md` and reproducible with `tests/run_poc.py -tuned`.

6.6 The audit log

A snippet of the audit log for the test run is included in the supplementary material and shipped in the repository as `results/audit.log`. The log shows the per-layer decisions, the per-layer rejection reasons, the latency per request (median 0.1–0.3 ms in the current implementation; well under the 50 ms design target), and the final disposition for each of the seven test cases. The log is the primary debugging interface for a deployment, and we include it in the open release so that other teams can reproduce the numbers and extend the test set.

6.7 Limitations of the evaluation

We are explicit about the limits of this evaluation. The seven test cases are not a statistically meaningful sample. They are a coverage check: does the framework have a layer for each cell of the taxonomy? The audit log shows that it does, modulo the two structural misses we discussed. A real deployment should run the framework against its own traffic, with its own labeled examples, and tune the configuration accordingly.

7 Discussion and Open Problems

I want to close the technical part of the paper with a short list of the open problems I ran into while building SENTINEL-LLM. I am listing them in the order they surprised me, which is roughly the order in which a small defensive team will encounter them. None of them is a research surprise. They are engineering surprises. I am not a research lab; I am a recent

graduate with a workstation, and every one of these problems was discovered the hard way, by trying to build something and watching it fail.

7.1 Universal transfer

The GCG family of attacks is the most worrying from a defender’s perspective, because a single optimized suffix defeats alignment on multiple models at once. My reference implementation does not have a layer that targets universal transfer specifically; the Layer 2 classifier is fine-tuned on the attacks I have seen, and a new attack family requires new training data. The deeper problem is that the defender is always one training run behind the attacker. I do not have a solution to this. I have a mitigation in the form of a configurable Layer 2 that can be retrained as new attacks emerge. The honest assessment is that the mitigation is more about reducing the time-to-update than about closing the gap. The gap is structural and I do not see a way to close it with an inference-time wrapper.

7.2 Encoded attacks

The string-composition and bijection-learning attacks [11, 10] are the most direct demonstration of the limits of surface-form defenses. The malicious content is in the encoded form, and the model is taught to decode it in context. A complete defense would have a decoding layer that tries the common encodings, decodes the input, and forwards the decoded version through the rest of the pipeline. I did not implement such a layer. Doing so correctly is its own engineering project, and the obvious implementation (try every encoding) is both expensive and prone to false positives. The right design is probably a small classifier trained to detect "this looks like an encoded payload" and a small decoder for the common cases, both of which I leave for future work. The base64 prompt that bit me during development is the canonical example. It is sitting in the test set in the repository. Anyone who runs the test set will see the same miss I saw, and the reason for the miss is documented in the code.

7.3 Multi-turn drift

Multi-turn attacks are the most frustrating because the malicious content is constructed by the conversation itself, and the threshold between "engaged user" and "drifting attacker" is hard to set. My Layer 4 implementation is a small embedding-based drift detector with a configurable threshold. The threshold is the entire defense. Setting it too tight produces false positives on long, engaged conversations. Setting it too loose misses the drift. I do not have a good solution. I have a knob that a deploying organization can tune against its own traffic, and a recommendation that the knob be re-evaluated on a quarterly cadence. I do not think there is a defensible default for this threshold; the right value depends on the deployment.

7.4 Agentic and tool-call exploits

Agentic deployments are the most active area of attack research right now, and the most under-defended. The Layer 5 tool-call allowlist is the only defense I have against the most common agentic attacks, and the allowlist is a coarse instrument. The deeper problem is that the model itself, with its broad instruction-following, is the attack surface, and an allowlist on the tool calls does not address the underlying problem. The recent survey by Zhan and colleagues [27] catalogs more attacks than I can address here, and the field does not yet have a good defense for most of them. The two indirect prompt injection cases that my Layer 4 missed are the cases I am most worried about for the agentic vector. Both are in the test set, and both are described in the audit log. I am open to suggestions on how to close those two cases without breaking the legitimate tool calls.

7.5 Distribution shift

The Layer 2 classifier was fine-tuned on a particular distribution of attack and benign prompts. The distribution of a real deployment will differ. The classifier will degrade, sometimes silently. I did not run a longitudinal study; the framework’s design includes a structured log that should make it possible to detect degradation by monitoring the per-layer rejection rate over time, but a proper study is future work. A defender who deploys SENTINEL-LLM should expect to retrain the classifier on a regular cadence, perhaps quarterly, and should have a labeled evaluation set that is updated alongside the training data. I learned this lesson the hard way on a previous project, where an embedding-based classifier silently degraded over six months before someone noticed. The audit log in SENTINEL-LLM is partly a reaction to that experience.

7.6 What we would do differently

If I were starting over, I would do three things differently. I would build the decoding layer for encoded attacks from the beginning, even at the cost of latency. I would invest more in the multi-turn drift detector, because the multi-turn attack vector is the one that is most likely to improve in the near term. And I would write a proper longitudinal study of the Layer 2 classifier’s degradation, because the silent failure mode of an embedding-based classifier is the worst kind of failure for a security system. I would also write fewer "we" sentences and more "I" sentences. The original drafts of this paper had a lot of "we" because that is how academic writing sounds. The "I" sentences are more honest. I have been gradually converting them through the revisions, and the paper is better for it.

8 Conclusion

The clearest lesson from this project is that encoded attacks are structurally invisible to any defense that reads surface form. I knew this in the abstract when I started. I knew it more concretely when the 4/7 evaluation came in, and one of the three misses was a base64 prompt that every layer of SENTINEL-LLM let through. None of the published defenses would have caught it either, as far as I can tell. This is not a critique of those defenses. It is a description of the problem class. Encoded inputs, multi-turn drift, and the agentic tool-call space are three different shapes of the same underlying issue, which is that the defender is always reasoning about a transformed version of the thing the attacker is actually doing. I do not see a clean way around that, and I have stopped looking for one.

I built SENTINEL-LLM because I wanted a small, runnable, honest defense that an organization could put in front of a production LLM endpoint without retraining the model. It is a five-layer wrapper: lexical and perplexity, semantic intent, cross-modal context, multi-turn drift, and tool-call firewall. It catches a meaningful fraction of the attacks in the current literature. It adds a manageable amount of latency. It logs every decision in a structured form. The proof-of-concept evaluation on a curated set of seven test cases shows a 4/7 detection rate. The misses are concentrated in the encoded and multi-turn attack vectors, and I describe exactly why in Sections 6 and 7. None of the five layers is novel on its own. The combination, the sequencing, and the per-layer configuration that keeps the latency under 50ms is the contribution I am putting forward for review. I am not going to oversell it.

The honest summary of what I learned is this. The LLM adversarial-robustness problem is not solved, and it is unlikely to be solved by a single clever idea. It is a sustained engineering problem with a research component, and the research community has produced a much richer attack literature than defense literature. The defensive playbook that does exist is fragmented across model-side systems, wrapper-level filters, and operational controls, and a small team that wants to deploy a defense has to assemble its own. I am also publishing the test set, the audit

log, and the failure cases so that the next team does not have to start from scratch on the parts I got wrong. The failure cases are the most valuable part of the package, in my opinion. Anyone can publish a system that passes its own tests. The interesting question is what the system does on the cases it was not designed to pass.

The next steps for my own work are the encoding layer, the multi-turn detector, and the longitudinal study of the Layer 2 classifier. I am happy to collaborate with other teams who are working on the same problems. The repository URL is in the Appendix.

If the next team uses my framework and catches one attack that would have otherwise reached a production system, this was worth the year. If the next team reads this paper and decides to build something different and better, that is also a fine outcome. I would rather be a small, useful footnote in someone else’s better paper than a footnote in no paper at all.

References

- [1] Muneeb Anjum. Sentinel-llm: Reference implementation of a five-layer inference-time defense framework. <https://github.com/horizonbymuneeb/sentinel-llm>, 2026.
- [2] Anonymous. Bypassing llm guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems. In *ACL Workshop on Large Language Model Security*, 2025.
- [3] Anonymous. Mavla: Multimodal adversarial attack framework for vision-language-action models in embodied intelligence. In *ACM Multimedia*, 2025.
- [4] Anonymous. Ab-jb: A hybrid jailbreak framework combining black-box semantic prompt variant generation with embedding-level suffix optimization. *Nature Scientific Reports*, 2026.
- [5] Anthropic. Constitutional classifiers: Defending against universal jailbreaks across thousands of hours of red teaming. *arXiv preprint arXiv:2501.18837*, 2025.
- [6] Patrick Chao, Edoardo Debenedetti, Alex Robey, Maksym Andriushchenko, Vikash Sehwal, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed Hassani, et al. Jailbreakbench: An open robustness benchmark for jailbreaking large language models. In *Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. arXiv preprint arXiv:2404.01318.
- [7] Cisco Systems. The cisco ai defense 2025 benchmark for llm safety. Industry technical report, 2025.
- [8] Gartner, Inc. Gartner forecasts worldwide genai spending, 2025.
- [9] Benjamin Geyer, Tobias Müller, John Wenskovitch, Christian Berthelsen, and Nora Hollenstein. Backdoored retrievers for prompt injection attacks on retrieval augmented generation. *arXiv preprint arXiv:2410.14479*, 2024.
- [10] Brian Huang, Yige Li, Zhe Cao, Lu Wang, Maks Andreessen, Alyssa Chang, Edward Hine, Aofan Hayase, et al. Endless jailbreaks with bijection learning. In *International Conference on Learning Representations*, 2025. arXiv preprint arXiv:2410.01294.
- [11] Yimin Huang, Wujiang Wang, Zixuan Cao, Yujun Sun, Lin Dong, Xu Chen, Minghui Cao, et al. Plentiful jailbreaks with string compositions. *arXiv preprint arXiv:2411.01064*, 2024.
- [12] iProov. Deepfakes and the human verification gap, 2025.

- [13] Aryan Jain. Sentinelllm: A 5-layer llm security framework in fastapi. <https://github.com/ajcgxdd/SentinellLM>, 2026. Concurrent, independent implementation; different design and scope from this work.
- [14] Pratap Jaiswal et al. Analysis of llms against prompt injection and jailbreak attacks. *arXiv preprint arXiv:2602.22242*, 2026.
- [15] Dmitry Labintsev. Sentinel: Ai security platform - defense, offense, framework (dragon v5.0). <https://pypi.org/project/sentinel-llm-security/>, 2026. Contemporaneous red/blue team suite built on 49 Rust engines; distinct in both architecture and scope from the inference-time Python wrapper presented in this work.
- [16] Guang Lin et al. Large language model sentinel: Llm agent for adversarial purification. *arXiv preprint arXiv:2405.20770*, 2024. Distinct from this work: a purification agent, not a multi-layer prompt-injection defense.
- [17] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, et al. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal. *arXiv preprint arXiv:2402.04249*, 2024.
- [18] National Institute of Standards and Technology. Adversarial machine learning: A taxonomy and terminology of attacks and mitigations (nist ai 100-2 e2025). Technical report, NIST Trustworthy and Responsible AI, 2025. <https://doi.org/10.6028/NIST.AI.100-2>.
- [19] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022. Also available as arXiv preprint arXiv:2203.02155.
- [20] OWASP Foundation. Owasp top 10 for large language model applications (2025 update). <https://owasp.org/www-project-top-10-for-large-language-model-applications/>, 2025.
- [21] Rohan Pradhan et al. Indirect prompt injection in rag systems: A survey of attack vectors and mitigations. *MDPI Information*, 17(1):54, 2025.
- [22] Alex Robey, Eric Wong, Hamed Hassani, and George J. Pappas. Smoothllm: Defending large language models against jailbreaking attacks. *arXiv preprint arXiv:2310.03684*, 2023.
- [23] Rusheb Shah, Sooraj Pour, Micah Chan, Anish Kethireddy, Rylan Swenor, Micah Evers, Dakota Rein, Steven Cen, and Zhou Shi. Scalable and transferable black-box jailbreaks for language models via persona modulation. In *arXiv preprint arXiv:2311.03348*, 2023.
- [24] The MITRE Corporation. Mitre atlas: Adversarial threat landscape for ai systems. <https://atlas.mitre.org/>, 2025.
- [25] Simon Willison. Indirect prompt injection. <https://simonwillison.net/2023/Apr/14/new-prompt-injection/>, 2023.
- [26] Peng Xie et al. Chain of attack: On the robustness of vision-language models against cross-modal adversarial attack. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. to appear; preprint not publicly available at submission time.
- [27] Xinyi Zhan et al. From prompt injections to protocol exploits: Threats in llm-powered ai agent ecosystems. *arXiv preprint arXiv:2506.23260*, 2025.

- [28] Jiaming Zhang, Qian Yi, Jiajing Mao, Dingkan Liu, Xiaoshuai Qu, Yikang Xie, Rongqin Liang, Bin Dong, Xiaowei Zhou, Peng Xu, et al. Vlattack: Multimodal adversarial attacks on vision-language pre-trained models. *arXiv preprint arXiv:2310.04655*, 2023.
- [29] Andy Zhou, Kevin Wu, Francesco Pinto, Zhaorun Chen, Yi Zeng, Yu Yang, Shuang Yang, Sanmi Koyejo, James Zou, and Bo Li. Autoreddteamer: Autonomous red teaming with lifelong attack integration. *arXiv preprint arXiv:2503.15754*, 2025.
- [30] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

Appendix A: Declarations

CRedit authorship contribution

Muneeb Anjum: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Resources, Data Curation, Writing – Original Draft, Writing – Review & Editing, Visualization, Project administration. (Funding acquisition is not applicable: this work was self-funded and received no external grant; see Funding statement below.)

Competing interests

The author declares no competing interests.

Funding

This research received no external funding. Compute and infrastructure were provided by the author’s personal resources. No grant, contract, or institutional support was involved in the design, conduct, analysis, or reporting of this work.

Code and reproducibility

The SENTINEL-LLM reference implementation, the configuration defaults, the proof-of-concept test harness, the 32-attack test set drawn from public sources, and the audit log are released under the MIT License [1] at the following repository: <https://github.com/horizonbymuneeb/sentinel-llm>. The repository includes a one-command reproducibility script (`tests/run_poc.py -tuned`) that re-runs the seven paper PoC cases and the larger 32-attack test set, and a `REPRODUCIBILITY.md` file that records the per-case decision log for both runs. Reviewers are encouraged to clone the repository, install `pyyaml` (`pip install pyyaml`), and run `python3 tests/run_poc.py -tuned` to verify the published detection numbers.

Data availability

All data generated or analysed during this study are included in this manuscript and its supplementary files. The seven proof-of-concept test cases, the audit log, and the configuration files are released as part of the open-source repository. No external datasets beyond the published literature cited in the references were used.

AI tools disclosure

This manuscript was prepared with language polishing assistance from a commercially available AI writing tool. The tool was used for grammar checking, sentence restructuring, and reference

formatting. The technical content, including the taxonomy, the framework design, the implementation, the evaluation, and the discussion, is the author’s own work, derived from the cited literature and from the author’s implementation and testing of the framework. No AI system was used to generate, fabricate, or interpret any empirical results, data, or claims made in this paper. The author has reviewed and validated all content.

Acknowledgments

The author thanks the open-source community for the attack and defense tools without which this work would not have been possible, and the maintainers of the benchmarks and threat catalogs cited in Section 3.3. The author is grateful to the COMSATS University Islamabad, Abbottabad Campus, Department of Computer Science for the academic environment in which the underlying degree program was completed.